

Programação Web

React



Prof^a. Jéssica Di Domênico



React JS

React JS

O que é?

- React é uma biblioteca **JavaScript** criado pelo Facebook (atual Meta) que é usado para criar interfaces de usuário (UI) em aplicativos web.
- React é uma biblioteca de código aberto;
- Comunidade ativa de desenvolvedores que contribuem com códigos e recursos para ajudar a melhorar a tecnologia;

React

Principais casos de uso

- criar aplicações web complexas que precisam ser atualizadas em tempo real. O React é ideal para esse tipo de aplicação porque ele permite que você atualize a interface de usuário de forma rápida e eficiente.
- criar aplicações web que precisam ser escaláveis e mantidas por equipes grandes. O React é uma biblioteca modular, o que significa que os componentes podem ser facilmente reutilizados e compartilhados entre diferentes partes da aplicação.

React

Componentes

- As aplicações são criadas através de componentes – composição;
- Construção de componentes complexos a partir de componentes mais simples;
- Composição também permite a criação de variações de componentes existentes;

JSX

O que é JSX?

- JSX (JavaScript XML) é uma extensão de sintaxe para JavaScript que permite escrever HTML dentro do código JavaScript.

```
// JSX
const elemento = <h1>Olá React!</h1>;

// Compilado para:
const elemento = React.createElement('h1', null, 'Olá React!');
```


JSX

O que é JSX?

Vantagens:

- Sintaxe familiar para quem conhece HTML
- Permite usar toda a lógica do JavaScript (condicionais, loops, etc.)
- React transforma JSX em chamadas de função `React.createElement()`

Fragmentos

<> </>

Problema: Funções em JavaScript só podem retornar um único valor. No React, isso significa que um componente só pode retornar um único elemento raiz.

Solução: Use Fragmentos (<> </>) para agrupar múltiplos elementos sem adicionar nós extras no DOM.

```
// ✗ Erro: dois elementos raiz
return (
  <h1>Título</h1>
  <p>Parágrafo</p>
);

// ✔ Correto: usando Fragment
return (
  <>
    <h1>Título</h1>
    <p>Parágrafo</p>
  </>
);
```

React

Componentes

```
function Titulo() {  
  return (  
    <div>  
      <h1>Olá mundo React. Meu primeiro componente!</h1>  
    </div>  
  );  
}  
  
export default Titulo;
```


React

Componentes

```
const Corpo = () => {  
  return (  
    <div>  
      <h2>Corpo da Página</h2>  
    </div>  
  )  
}  
  
export default Corpo;
```

React

Componentes

```
import Titulo from "../Titulo";
import Corpo from "../Corpo";

function App() {

  return (
    <div>
      <Titulo />
      <Corpo />
    </div>
  );
}

export default App;
```

Componentes

Props

- Dados podem ser manipulados em um componente pai, e eles podem ser passados por props para os componentes filhos;
- O objeto props é **somente leitura**, não é possível alterar os valores passados aos componentes filhos com uso de props;
- Os dados passados por props podem ser números, string, arrays, objetos, funções, etc...

Props

```
// Componente PAI
function App() {
  const dados = ['React', 'NPM', 'JSX'];
  return <Filho nome="João" idade={25} tecnologias={dados} />;
}

// Componente FILHO
function Filho(props) {
  return (
    <div>
      <h1>Nome: {props.nome}</h1>
      <h2>Idade: {props.idade}</h2>
      <ul>
        {props.tecnologias.map(tech => <li key={tech}>{tech}</li>)}
      </ul>
    </div>
  );
}
```

Hooks

O que são?

- Hooks são funções especiais do React que permitem "conectar" recursos do React a componentes funcionais.
- Permitem que você use o state e outros recursos sem escrever classes, em componentes funcionais;

Regras dos Hooks:

1. Só podem ser chamados no nível superior do componente (não dentro de loops, condicionais ou funções aninhadas).
2. Só podem ser chamados dentro de componentes funcionais ou outros hooks.

Hooks

O que são?

Hook	Para que serve
useState	Gerenciar estados locais
useEffect	Executar efeitos colaterais (API, timers, etc.)

Hooks

useState

useState permite adicionar estado a componentes funcionais.

```
const [estado, setEstado] = useState(valorInicial);
```

useState

Como funciona

Parte	Descrição
estado	Valor atual do estado
setEstado	Função para atualizar o estado
useState(0)	Inicia o estado com valor 0

useState

Exemplo Prático

```
const [contador, setContador] = useState(0);  
  
// Quando o botão é clicado:  
setContador(contador + 1); // Atualiza e RE-RENDERIZA o componente  
  
// ⚠ O componente é renderizado novamente com o novo valor!
```

Hooks

useEffect

useEffect é um Hook do React que permite executar efeitos colaterais em componentes funcionais.

São ações que ocorrem além da renderização do componente, como:

- Buscar dados de uma API
- Configurar timers (setTimeout, setInterval)
- Manipular o DOM diretamente
- Inscrever-se em eventos (scroll, resize, etc.)
- Salvar dados no localStorage

```
useEffect(() => {  
  // Código do efeito (executa após a renderização)  
}, [/* dependências */]);
```


useEffect

O que é?

Parte	Descrição
Função do efeito	Código que será executado após a renderização. Pode conter qualquer lógica assíncrona ou síncrona.
Array de dependências	Lista de valores que, quando mudarem, farão o efeito executar novamente.
<code>useState(0)</code>	Inicia o estado com valor 0

useEffect

Exemplo prático

```
const [resultado, setResultado] = useState(0);  
const [historico, setHistorico] = useState([]);  
  
// NOVO: useEffect salva cada resultado no histórico  
useEffect(() => {  
  if (resultado !== 0) {  
    setHistorico([...historico, resultado]);  
  }  
}, [resultado]); // Executa quando resultado muda
```

Renderização

como o React faz isso?

Virtual DOM → "fotografia" da interface na memória

Diffing → compara antes/depois e só atualiza o que mudou

Re-renderização seletiva → componentes que não mudaram não são recriados

Renderização condicional → só renderiza o que está visível

Renderização Condicional

Em React, podemos usar JavaScript puro para decidir o que renderizar:

```
// 1. Operador && (AND)
{mostrar && <p>Este texto aparece se 'mostrar' for true</p>}

// 2. Operador ternário (if/else)
{resultado > 10 ? <h1>Aprovado</h1> : <h1>Reprovado</h1>}
```


Re-renderização no React

Quando o React re-renderiza um componente?

1. Quando seu estado (state) muda;
2. Quando suas props mudam;
3. Quando seu componente pai re-renderiza.

```
function Contador() {  
  const [contador, setContador] = useState(0);  
  
  console.log("Renderizando!"); // ✅ Aparece a cada clique  
  
  return (  
    <>  
      <h1>{contador}</h1>  
      <button onClick={() => setContador(contador + 1)}>+</button>  
    </>  
  );  
}
```

Importante: Sempre que setContador é chamado, o componente é renderizado novamente → a função inteira executa de novo!

CÂMPUS
Passo Fundo

